데이터베이스(SQL)

7장. SQL JOIN

학습 목표

1) JOIN이 필요한 이유를 설명할 수 있다.
2) JOIN의 본질을 이해할 수 있다.
3) PK와 FK를 이용해 테이블을 연결할 수 있다.
4) INNER JOIN과 OUTER JOIN을 구분하여 사용할 수 있다.
5) ON과 WHERE의 차이를 설명할 수 있다.
6) JOIN 후 행이 증가하는 이유를 설명할 수 있다.
7) SELF JOIN을 사용할 수 있다.
8) 3개 이상의 테이블을 연결할 수 있다.
9) JOIN 전에 각 테이블의 한 행이 무엇을 뜻하는지(행 단위)를 확인할 수 있다.
10) PdM 데이터에서 machineid와 datetime을 이용해 설비·센서·이벤트를 안전하게 연결할 수 있다.

공식 문서: 테이블 표현식 (JOIN)

전체 흐름

1) JOIN이 필요한 이유 → 2) JOIN이란 → 3) JOIN 종류 → 4) SELF JOIN과 여러 테이블

5) ON과 WHERE → 6) 집합 연산

01 JOIN의 기초

JOIN이 필요한 이유

관계형 데이터베이스는 데이터를 여러 테이블로 나누어 저장한다.

customers

customer_id	company_name	city
ALFKI	Alfreds Futterkiste	Berlin

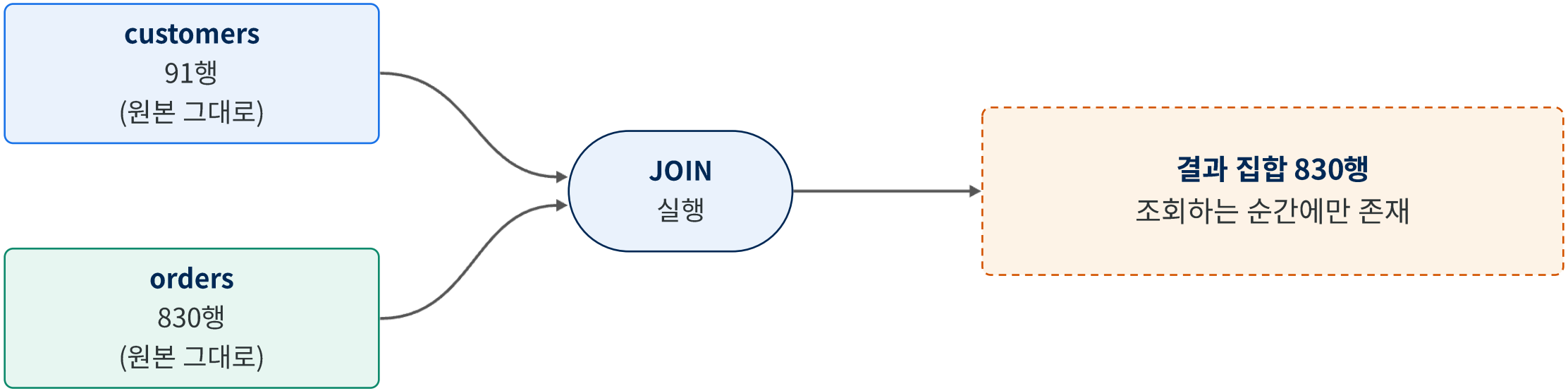
orders (ALFKI 주문 중 일부)

order_id	customer_id	order_date
10643	ALFKI	1997-08-25
10692	ALFKI	1997-10-03

> “고객 회사명과 주문일자를 함께 보고 싶다”고 하면 두 테이블을 합쳐야 한다. 이때 사용하는 기능이 JOIN이다.

JOIN 이란

두 개 이상의 테이블을 결합 조건에 따라 연결하여, 관련된 데이터를 하나의 조회 결과로 나타내는 연산이다.



PK와 FK로 연결하기

JOIN은 보통 Primary Key와 Foreign Key를 기준으로 한다.

PK/FK가 없어도 JOIN할 수 있다

PK와 FK가 가장 분명한 연결 기준이다. PdM처럼 FK가 없어도 같은 대상을 뜻하는 컬럼(machineid)으로 연결할 수 있다.

중요한 것은 컬럼 이름이 같은지가 아니라, 두 컬럼이 같은 대상을 가리키는지 확인하는 것이다.

연결 조건



ON customers.customer_id = orders.customer_id 같은 고객 번호를 가진 행끼리 연결

JOIN 기본 문법

문법

```
SELECT 컬럼목록
FROM 테이블A
JOIN 테이블B
ON 테이블A.컬럼 = 테이블B.컬럼;
```

기본 예제

```
SELECT
    c.customer_id,
    c.company_name,
    o.order_id,
    o.order_date
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
ORDER BY o.order_date, o.order_id
LIMIT 10;
```

순서	동작
1	FROM / JOIN - 테이블을 구성
2	ON - JOIN에서 어떤 행을 매칭할지 결정
3	WHERE - 구성된 행을 필터링
4	SELECT - 필요한 식과 컬럼을 출력

별칭(Alias) 사용

테이블명이 길면 별칭을 쓴다

```
-- 별칭 없음 (길고 반복적)
SELECT customers.company_name, orders.order_date
FROM customers
JOIN orders ON customers.customer_id = orders.customer_id;

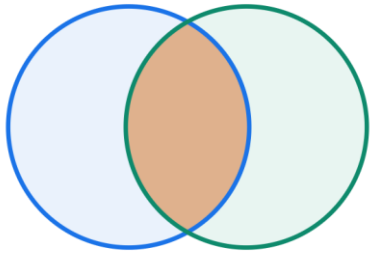
-- 별칭 사용 (간결)
SELECT c.company_name, o.order_date
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

JOIN의 처리 순서



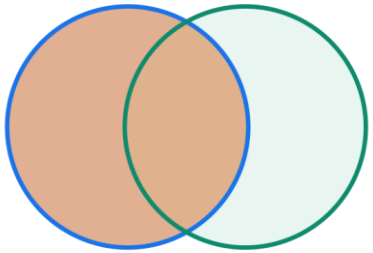
02 JOIN의 종류

JOIN 종류



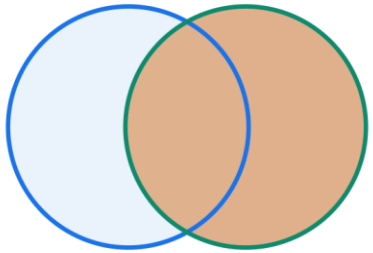
INNER JOIN

양쪽에 모두 있는 행만



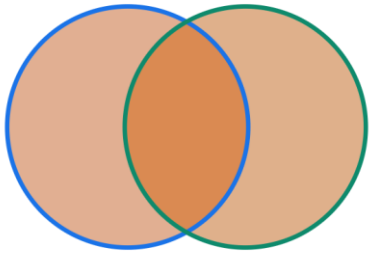
LEFT JOIN

왼쪽 행 전부 + 일치하는 오른쪽 행



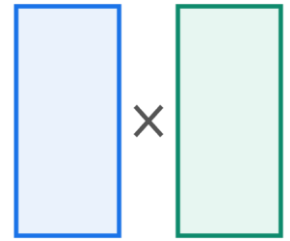
RIGHT JOIN

오른쪽 행 전부 + 일치하는 왼쪽 행



FULL JOIN

양쪽 행 전부



CROSS

모든 조합

INNER JOIN

양쪽 테이블에 **모두 존재하는** 데이터만 반환한다.

customers

customer_id	company_name
ALFKI	Alfreds
PARIS	Paris Spec

orders

order_id	customer_id
10643	ALFKI

```
SELECT
        c.customer_id,
        c.company_name,
        o.order_id
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

결과

customer_id	company_name	order_id
ALFKI	Alfreds	10643

주문이 없는 PARIS는 결과에서 제외된다.

JOIN과 INNER JOIN

JOIN만 쓰면 자동으로 INNER JOIN이다.

```
-- 아래 둘은 완전히 동일
FROM customers c JOIN orders o ON ...
FROM customers c INNER JOIN orders o ON ...
```

활용

상황	예시
주문한 고객만 조회	customers + orders
판매된 제품만 조회	products + order_details
담당자가 있는 주문만	orders + employees

LEFT JOIN

왼쪽 테이블의 데이터는 **모두 유지**한다. 오른쪽에 매칭되는 데이터가 없으면 NULL로 채운다.

```
SELECT
        c.customer_id,
        c.company_name,
        o.order_id
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

결과

customer_id	company_name	order_id
ALFKI	Alfreds	10643
PARIS	Paris Spec	NULL

주문이 없어도 고객 정보는 유지된다.

공식 문서: OUTER JOIN

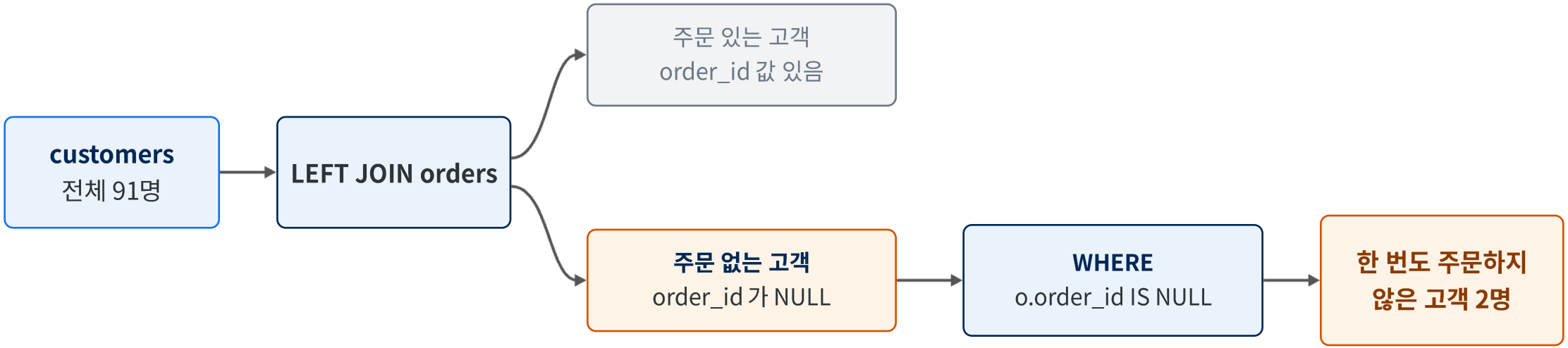
LEFT JOIN과 IS NULL

LEFT JOIN으로 **누락 데이터 찾기**

```
-- 한 번도 주문하지 않은 고객 찾기
SELECT
        c.customer_id,
        c.company_name
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

LEFT JOIN + IS NULL 패턴의 원리

1) LEFT JOIN으로 모든 고객 유지
2) 주문이 없는 고객은 order_id가 NULL
3) WHERE로 NULL만 필터링
↓ 주문 이력이 없는 고객만 남음



RIGHT JOIN

오른쪽 테이블 데이터를 모두 유지한다.

```
SELECT
        c.company_name,
        o.order_id
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

RIGHT JOIN은 테이블 순서를 바꿔 LEFT JOIN으로 쓸 수 있다. 항상 "왼쪽이 기준"으로 읽을 수 있어 LEFT JOIN으로 통일하면 읽기 쉽다.

```
-- RIGHT JOIN
FROM customers c RIGHT JOIN orders o ON ...

-- 같은 결과, LEFT JOIN으로 표현 (권장)
FROM orders o LEFT JOIN customers c ON ...
```

공식 문서: JOIN 타입

FULL JOIN

양쪽 테이블의 데이터를 모두 유지한다.

```
SELECT
        c.company_name,
        o.order_id
FROM customers c
FULL JOIN orders o
ON c.customer_id = o.customer_id;
```

공식 문서: JOIN 타입

CROSS JOIN

두 테이블의 모든 행을 서로 조합하여 카테시안 곱(Cartesian Product)을 생성하는 조인 연산이다.

```
SELECT
c.company_name,
p.product_name
FROM customers c
CROSS JOIN products p;
```

카테시안 곱(Cartesian Product) - 두 집합의 모든 원소를 각각 하나씩 짝지어 만들 수 있는 모든 순서쌍의 집합을 의미

공식 문서: CROSS JOIN

CROSS JOIN 결과 크기

A × B	b1	b2	b3	b4
a1	a1-b1	a1-b2	a1-b3	a1-b4
a2	a2-b1	a2-b2	a2-b3	a2-b4
a3	a3-b1	a3-b2	a3-b3	a3-b4

3행 × 4행 = 12행 (원리 예)

고객 91명 × 제품 77개 = 7,007행

결과 행 수는 항상 두 테이블 행 수의 곱이다

JOIN 종류 한 줄 요약

JOIN	남는 것	비는 곳
INNER	양쪽에 모두 매칭되는 행	없음 (매칭 안 되면 행 자체가 제외)
LEFT	왼쪽 전체	오른쪽 매칭 없으면 오른쪽 컬럼이 NULL
RIGHT	오른쪽 전체	왼쪽 매칭 없으면 왼쪽 컬럼이 NULL
FULL	양쪽 전체	매칭 없는 쪽 컬럼이 NULL
CROSS	모든 조합 (A행수 × B행수)	조건 자체가 없음

03 SELF JOIN과 여러 테이블

SELF JOIN

같은 테이블을 자기 자신과 연결한다. 계층 구조나 자기 참조 관계에 사용한다.

Northwind 예제 - 직원과 상사 employees 테이블에는 reports_to 컬럼이 있다. 상사의 employee_id를 담고 있다.

```
SELECT
        e.employee_id, -- 직원 번호
        e.first_name || ' ' || e.last_name AS employee_name, -- 직원 이름
        m.first_name || ' ' || m.last_name AS manager_name -- 상사 이름
FROM employees e
LEFT JOIN employees m
ON e.reports_to = m.employee_id
ORDER BY e.employee_id;
```

공식 문서: 테이블 별칭

SELF JOIN — 같은 테이블로 직원과 상사 연결

employees 테이블을 두 번 참조하되, e는 직원 · m은 상사 로 구분한다.

employees e | 직원

employee_id	직원 이름	reports_to (상사 ID)
1	Nancy Davolio	2
2	Andrew Fuller	NULL
3	Janet Leverling	2

상사 ID → 직원 ID

employees m | 상사

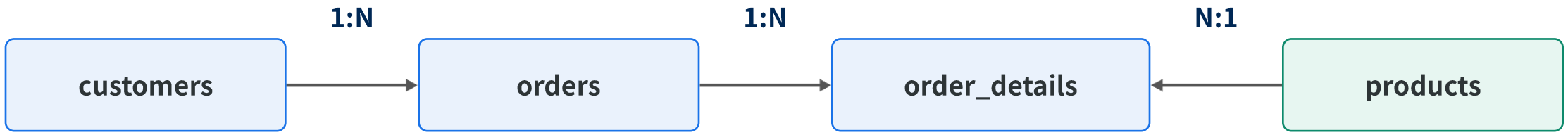
employee_id	직원 이름	reports_to (상사 ID)
1	Nancy Davolio	2
2	Andrew Fuller	NULL
3	Janet Leverling	2

JOIN 조건: e.reports_to = m.employee_id (직원 의 상사 ID와 상사 의 직원 ID를 비교)

LEFT JOIN 조회 결과 — 상사가 없어도 직원은 모두 표시 (3행)

직원 ID	직원 이름 (e)	상사 ID (e.reports_to)	상사 이름 (m)
1	Nancy Davolio	2	Andrew Fuller
2	Andrew Fuller	NULL	NULL
3	Janet Leverling	2	Andrew Fuller

여러 테이블 JOIN — Northwind 관계 구조



customers에서 orders, order_details를 거쳐 products까지 관계를 따라 차례로 연결한다.

4개 테이블 JOIN

```
SELECT
    c.company_name, -- 고객사
    o.order_id, -- 주문 번호
    o.order_date, -- 주문일자
    p.product_name, -- 제품명
    od.quantity, -- 수량
    od.unit_price, -- 단가
    ROUND((od.quantity * od.unit_price)::NUMERIC, 2) AS amount -- 금액(수량 × 단가)
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
JOIN order_details od
ON o.order_id = od.order_id
JOIN products p
ON od.product_id = p.product_id
ORDER BY o.order_date DESC, o.order_id, p.product_id
LIMIT 20;
```

여러 테이블 JOIN 작성 순서

1) 기준 테이블을 FROM에 놓는다

2) 관계를 따라 하나씩 JOIN을 추가한다

3) 각 JOIN마다 결과를 확인한다

JOIN 후 데이터가 증가하는 이유 — 상황

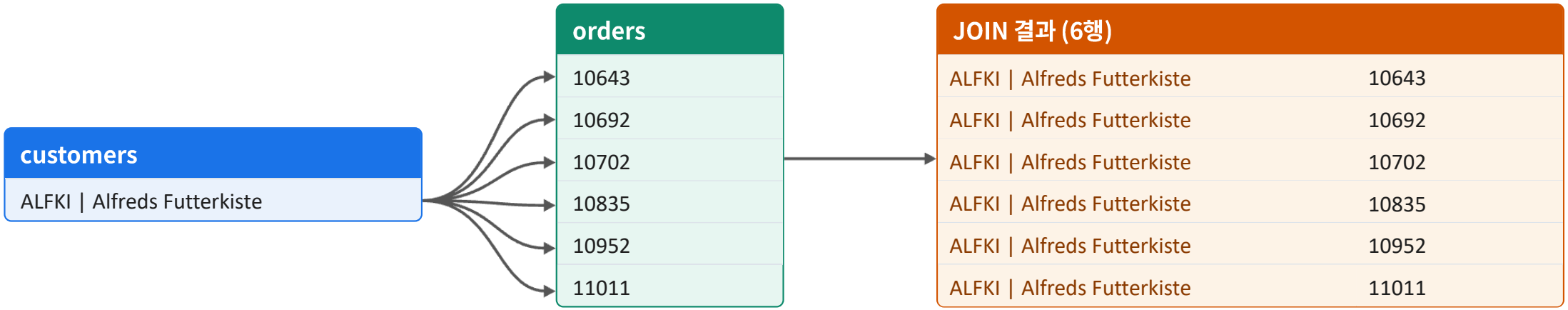
customers (ALFKI 1행)

customer_id	company_name
ALFKI	Alfreds Futterkiste

orders (ALFKI의 주문 6행)

order_id	customer_id
10643	ALFKI
10692	ALFKI
10702	ALFKI
10835	ALFKI
10952	ALFKI
11011	ALFKI

JOIN 결과 (6행)



고객 1행 × 주문 6건 = 결과 6행

원인 1:N 관계에서 1의 데이터가 N의 개수만큼 반복된다.

JOIN 전후 행 수 검증 습관

JOIN 전후 행 수를 항상 확인한다.

```
-- JOIN 전
SELECT COUNT(*) FROM orders; -- 830

-- JOIN 후
SELECT COUNT(*)
FROM orders o
JOIN order_details od ON o.order_id = od.order_id; -- 2155
```

04 ON과 WHERE

ON과 WHERE의 핵심 차이

절	적용 시점	역할
ON	JOIN 하는 중	어떻게 연결할지
WHERE	JOIN 완료 후	결과를 어떻게 거를지

공식 문서: JOIN 타입 (조인 조건)

INNER JOIN에서는 같은 결과

```
-- 단순한 행 필터 조건을 옮긴 경우 결과 집합은 동일
FROM a JOIN b ON a.id = b.id AND b.status = 'Y'
FROM a JOIN b ON a.id = b.id WHERE b.status = 'Y'
```

LEFT JOIN에서는 결과가 완전히 다르다

ON에 조건을 넣은 경우

-모든 고객 유지, 1997년 1월 1일 이후 주문만 연결

```
SELECT
        c.company_name,
        o.order_id,
        o.order_date
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
AND o.order_date >= '1997-01-01';
```

결과:
모든 고객이 조회되며, 1997년 1월 1일 이후의 주문이 없는 고객은 주문 관련 컬럼이 NULL로 표시된다.

WHERE에 조건을 넣은 경우

-1997년 1월 1일 이후 주문이 있는 고객만 조회

```
SELECT
        c.company_name,
        o.order_id,
        o.order_date
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_date >= '1997-01-01';
```

결과: LEFT JOIN이 사실상 INNER JOIN처럼 동작
1997년 1월 1일 이후의 주문이 있는 고객만 조회되며, 해당 기간에 주문이 없는 고객은 결과에서 제외된다.

WHERE 조건흐름

1) LEFT JOIN 실행 → 모든 고객 유지, 주문 없으면 NULL

↓

2) WHERE order_date >= '1997-01-01' 적용

↓

3) NULL은 비교 조건을 만족하지 못함

↓

4) 주문 없는 고객이 제거됨

LEFT JOIN — 조건을 ON에 두느냐 WHERE에 두느냐

묻고 싶은 것: 모든 고객의 1997년 주문 현황

ON 에 조건

```
LEFT JOIN orders o
ON c.customer_id = o.customer_id
AND o.order_date >= '1997-01-01'
```

연결하면서 조건을 함께 본다

1997년 주문이 있는 고객 → 주문 정보가 붙는다
1997년 주문이 없는 고객 → 행은 남고 order_id 는 NULL
고객 91명 전부 결과에 나온다

의도한 대로 나온다

WHERE 에 조건

```
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_date >= '1997-01-01'
```

연결이 다 끝난 뒤에 거른다

주문 없는 고객은 order_date 가 NULL 이 된다
NULL 은 어떤 비교 조건도 만족하지 못한다
그 고객들이 결과에서 사라진다

INNER JOIN 과 같아진다

ON 은 “어떻게 연결할지”, WHERE 는 “연결 결과를 어떻게 거를지”를 정한다. INNER JOIN 에서는 차이가 없다.

05 집합 연산

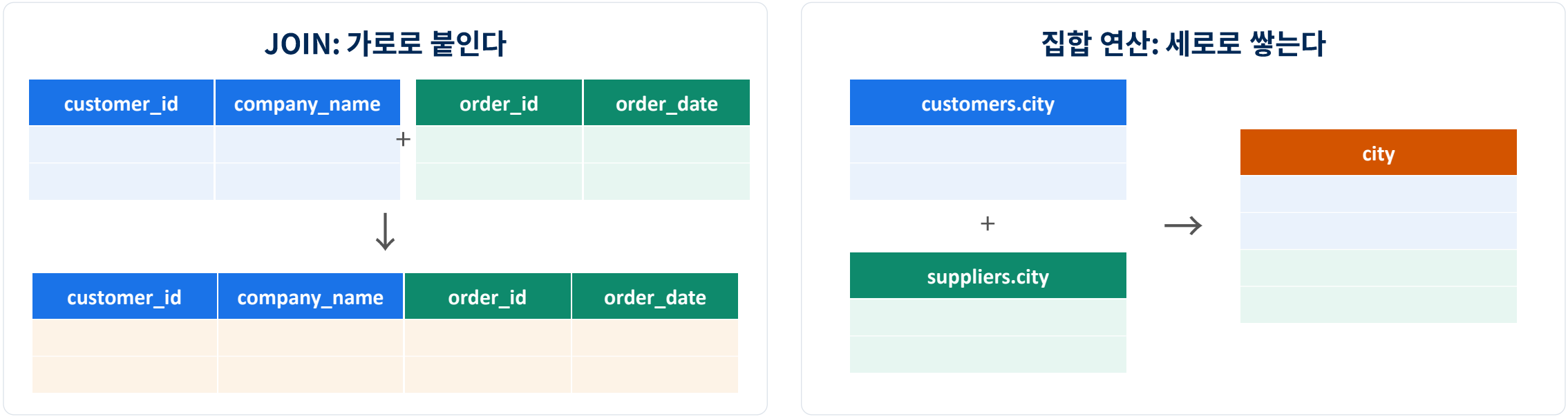
JOIN과 집합 연산의 차이 — 가로 vs 세로

JOIN은 두 테이블을 **가로로** 붙여 컬럼을 늘린다. 집합 연산은 두 조회 결과를 **세로로** 이어 행을 늘리거나 비교한다.

구분	JOIN	집합 연산
방향	가로 (컬럼 추가)	세로 (행 결합·비교)
연결 기준	ON 조건 (키 값)	위치 (컬럼 순서)
결과 컬럼	양쪽 컬럼 합침	첫 SELECT의 컬럼 구조

공식 문서: 질의 결합 (UNION, INTERSECT, EXCEPT)

JOIN과 집합 연산의 방향



JOIN의 연결 기준은 ON 조건(키 값), 집합 연산의 기준은 컬럼 위치

집합 연산과 사용 규칙

연산	의미	중복 처리
UNION	합집합 (양쪽 모두)	중복 제거
UNION ALL	합집합	중복 유지 (중복 제거 단계 없음)
INTERSECT	교집합 (양쪽에 다 있는 것)	중복 제거
EXCEPT	차집합 (첫 쪽에만 있는 것)	중복 제거

사용 규칙
각 SELECT의 **컬럼 수가 같고, 대응 위치의 타입이 호환**되어야 한다.
컬럼 이름은 첫 번째 SELECT를 따른다.
ORDER BY는 전체 결과에 대해 맨 마지막에 한 번만 쓴다.

집합 연산 4종

위쪽 SELECT — 고객 도시 (customers.city)

Berlin London Seoul

아래쪽 SELECT — 공급업체 도시 (suppliers.city)

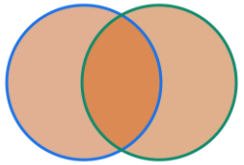
London Paris

UNION

합집합 · 중복 제거

4행

Berlin London Paris Seoul

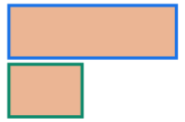


UNION ALL

합집합 · 중복 유지

5행

Berlin London Seoul London Paris

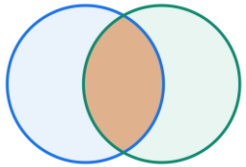


INTERSECT

교집합

1행

London

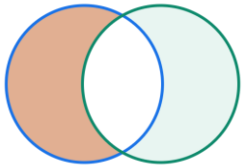


EXCEPT

차집합

2행

Berlin Seoul



앞쪽테이블에만 있는 것을 남긴다. 아래쪽에도 있는 London은 뺀다.

UNION과 UNION ALL

```
-- 고객 도시와 공급업체 도시를 하나의 목록으로
SELECT city FROM customers
UNION
SELECT city FROM suppliers
ORDER BY city;
-- 중복 제거됨: 양쪽에 다 있는 London은 한 번만 나온다
-- 결과: 93행 (고객 도시 69개 + 공급업체 도시 29개 - 양쪽에 다 있는 도시 5개)

SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
-- 중복 유지: 91 + 29 = 120행
```

INTERSECT와 EXCEPT

INTERSECT - 양쪽에 다 있는 것

```
-- 고객도 있고 공급업체도 있는 도시
SELECT city FROM customers
INTERSECT
SELECT city FROM suppliers
ORDER BY city;
-- 결과: 5개 도시 (Berlin, London, Montréal, Paris, Sao Paulo)
```

EXCEPT - 위쪽 결과에만 있는 것

```
-- 주문 이력이 없는 고객 찾기 (LEFT JOIN + IS NULL과 같은 문제, 다른 방법)
SELECT customer_id FROM customers
EXCEPT
SELECT customer_id FROM orders;
-- 결과: 2건 (FISSA, PARIS)
```

집합 연산 주의 — 컬럼 수가 다르면 오류

```
-- 오류 발생 코드
SELECT customer_id, company_name FROM customers
EXCEPT
SELECT customer_id FROM orders;
-- ERROR: each EXCEPT query must have the same number of columns
```

06 제조 데이터 JOIN

테이블별 행 단위

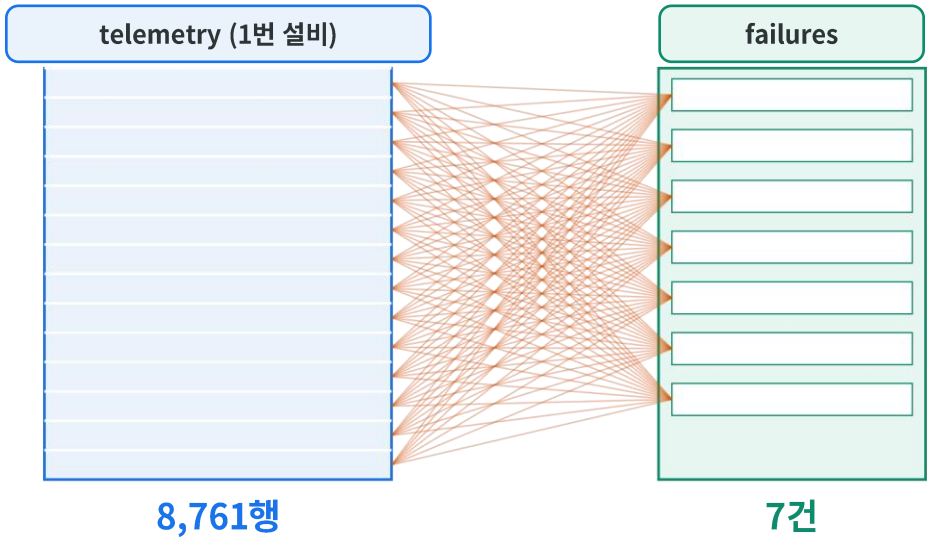
테이블	한 행의 의미	주요 연결 컬럼
pdm.pdm_machines	설비 1대	machineid
pdm.pdm_telemetry	설비 1대의 특정 시간대 센서 평균	machineid, datetime
pdm.pdm_errors	특정 설비에서 발생한 에러 이벤트 1건	machineid, datetime, errorid
pdm.pdm_maint	특정 설비의 부품 정비/교체 이벤트 1건	machineid, datetime, comp
pdm.pdm_failures	특정 설비의 부품 고장 이벤트 1건	machineid, datetime, failure

행 단위(grain)는 테이블의 한 행이 무엇을 뜻하는지다

> 행 단위가 다른 테이블을 machineid만으로 연결하면, 한 설비의 수많은 센서 행이 그 설비의 모든 이벤트 행과 조합되어 행 수가 크게 증가한다.

연결 키에 따른 행 수 — 분석 단위 비교

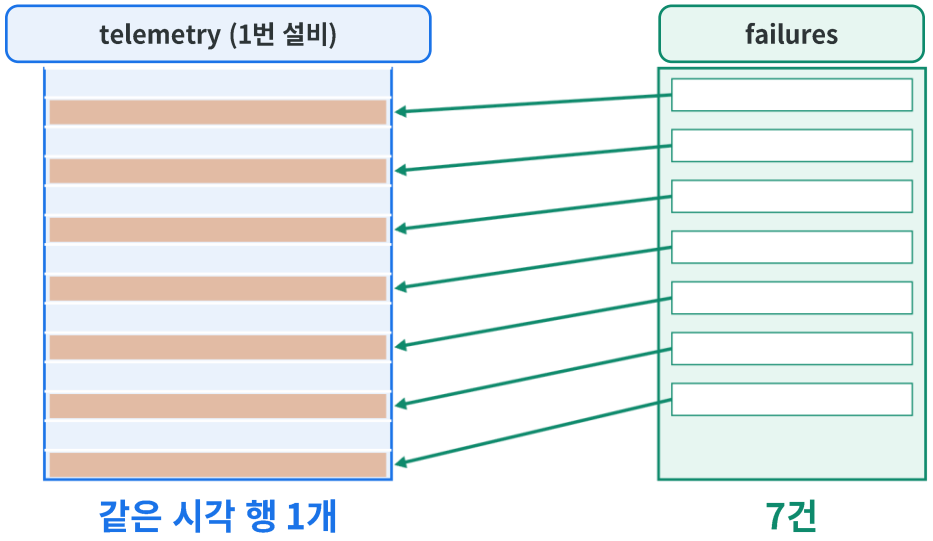
machineid만 연결



8,761 × 7 = 61,327행 (1번 설비)

전체 6,667,121행

machineid + datetime 연결



고장 1건 → 같은 시각 센서 1행

전체 761행 (고장 건수와 같음)

제조 JOIN 체크 순서

1) **질문의 분석 단위를 정한다.**
   예: 설비 1대 / 시간 1개

2) **각 테이블의 행 단위를 확인한다.**

3) **연결키를 정한다.**
   설비 정보 → machineid
   같은 시점 이벤트 → machineid + datetime

4) **JOIN 전후 COUNT(*)를 확인한다.**

5) **행이 늘었다면**
   정상적인 1:N인지,
   잘못된 다대다 곱셈인지 구분한다.

정리

원칙	이유
JOIN 조건을 반드시 확인한다	ON을 빠뜨리면 오류가 나고, 조건이 틀리면 행이 폭증한다
PK/FK와 업무 키 의미를 확인한다	같은 대상을 나타내는 컬럼으로 연결해야 한다
테이블 별칭을 사용한다	가독성이 크게 좋아진다
컬럼 앞에 별칭을 붙인다	어느 테이블 컬럼인지 명확해진다
필요한 컬럼을 명시한다	결과 구조가 명확하고 불필요한 데이터 처리를 줄이기 쉽다
JOIN 후 행 수를 확인한다	의도치 않은 증가를 잡아낸다
집계 시 DISTINCT를 고려한다	중복 집계를 방지한다

자주 발생하는 오류 1 — JOIN 조건 누락

```
-- JOIN에 ON이 없으면 문법 오류
SELECT * FROM customers c JOIN orders o;
-- ERROR: syntax error at or near ";"
-- (한국어 Windows: 오류: 구문 오류, ";" 부근)

-- 모든 조합을 의도한다면 명시적으로 CROSS JOIN
SELECT *
FROM customers c
CROSS JOIN orders o;
```

자주 발생하는 오류 2 — 컬럼명 모호성

```
-- 오류: customer_id가 양쪽에 다 있음 (주석을 풀면 오류가 난다)
-- SELECT customer_id
-- FROM customers c
-- JOIN orders o ON c.customer_id = o.customer_id;
-- ERROR: column reference "customer_id" is ambiguous
-- (한국어 Windows: 오류: 칼럼 참조 "customer_id" 가 모호합니다.)

-- 해결: 별칭 명시
SELECT c.customer_id
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

자주 발생하는 오류 3, 4

오류 3 - 잘못된 컬럼 연결

```
ON c.customer_id = o.order_id -- 잘못된 연결
ON c.customer_id = o.customer_id -- 올바른 연결
```

오류 4 - LEFT JOIN인데 WHERE로 필터링

```
-- 의도와 다르게 INNER JOIN처럼 동작
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '1997-01-01'
```

핵심 정리 (1/2)

1) JOIN은 테이블을 물리적으로 합치는 것이 아니라 조회 시점에 논리적으로 연결하는 것이다.
2) PK와 FK로 연결하는 것이 기본이고, PdM은 machineid + datetime으로 연결한다.
3) INNER JOIN은 양쪽에 모두 있는 데이터만 반환한다.
4) LEFT JOIN은 왼쪽 테이블을 모두 유지하고 없으면 NULL로 채운다.
5) LEFT JOIN + IS NULL은 누락 데이터를 찾는 대표 패턴이다.
6) RIGHT JOIN은 테이블 순서를 바꿔 LEFT JOIN으로 쓰는 것이 좋다.
7) CROSS JOIN은 모든 조합을 만든다. 결과는 두 테이블 행 수의 곱이다.

핵심 정리 (2/2)

8) SELF JOIN은 같은 테이블을 별칭을 다르게 해서 연결한다.

9) 1:N 관계에서 JOIN하면 행 수가 증가할 수 있다.

10) JOIN 후 고유 개체 수는 COUNT(DISTINCT)로 센다.

11) ON은 연결 조건, WHERE는 결과 필터링이다.

12) LEFT JOIN에서 WHERE에 오른쪽 테이블 조건을 넣으면 INNER JOIN처럼 동작한다.

13) PdM에서는 JOIN 전 각 테이블의 행 단위를 확인하고, 같은 시점의 센서·이벤트 연결에는 machineid + datetime을 사용한다.